For brevity, we show only part of the script. The full script is available with the book's files.

> **NOTE** To avoid having too many domain models and messing with sample data, we use the same movie data in chapters 8 and 9.

With an understanding of search mechanics and sample data in hand, let's turn to the fundamentals of search.

8.4 Search fundamentals

Now that we know the inner workings of search, let's look at the search API and how to invoke the engine to carry out search queries. Elasticsearch exposes a `_search` endpoint to communicate with it to execute search queries. Let's look at the endpoint in detail.

8.4.1 The _search endpoint

Elasticsearch provides RESTful APIs to query data: a `_search` endpoint, to be specific. We use `GET/POST` to invoke this endpoint, passing query parameters along with the request or a request body. The queries we construct depend on the type of data we are searching for. There are two ways of accessing the search endpoint:

- *URI request*—We pass the search query along with the endpoint as parameters to the query. For example, `GET movies/_search?q=title:Godfather` fetches all movies matching the word *Godfather* in the title (*The Godfather* trilogy, for example).
- *Query DSL*—Elasticsearch implements a DSL for search. Search criteria wrapped in a JSON body are sent to the server along with the request URL. The result is wrapped in a JSON object as well. We can provide a single query or combine multiple queries, depending on our requirements. (Query DSL is also the mechanism to send aggregate queries to the engine. We look more at aggregations later in the chapter.) Here's an example of the same requirement to fetch all movies with the word *Godfather* in the `title` field:

```
GET movies/_search
{
  "query": {
    "match": {
      "title": "Godfather"
    }
  }
}
```

Although both approaches are useful in their own ways, Query DSL is powerful and feature-rich. It is a first-class querying mechanism, and it's easier to write complex criteria using Query DSL than with the URI request mechanism. We look at various invocations in this chapter and chapters 9 through 12 and work more extensively with Query DSL than with the URI request method.